

Phase 1 Notes: Foundations of Multi-Way Search Trees and 2-3 Trees

0. Big Idea of Phase 1

Phase 1 teaches the foundation for balanced search trees.

A normal Binary Search Tree stores **one key per node**. A 2-3 tree allows a node to store **one or two keys**, so the tree can stay shorter and balanced.

The main goal is to understand this idea:

A 2-3 tree is like a Binary Search Tree, but each node can make more than two choices.

This helps the tree avoid becoming a long chain.

1. Binary Search Tree Review

1.1 What is a Binary Search Tree?

A **Binary Search Tree**, or **BST**, is a tree where each node stores one key.

Each node can have:

- one key
- one left child
- one right child

Example:

```
    30
   /
  20  40
```

The search rule is:

Smaller values go left.
Larger values go right.

So if we search for 40 :

40 > 30, so go right.
Found 40.

1.2 Why can a normal BST become slow?

A normal BST does not force itself to stay balanced.

If we insert sorted data like this:

10, 20, 30, 40, 50

we may get a tree like this:

```
10
 20
   30
    40
     50
```

This is no longer shaped like a good tree. It is shaped like a linked list.

In that case, searching may take $O(n)$ time because we may have to walk through many nodes one by one.

1.3 The problem Phase 1 wants to solve

The problem is:

How can we keep a search tree balanced so searching, inserting, and deleting stay fast?

A 2-3 tree solves this by allowing nodes to store more than one key and by forcing all leaves to stay at the same level.

2. Multi-Way Search Trees

2.1 What is a multi-way search tree?

A **multi-way search tree** is a search tree where one node can have more than two children.

A normal BST node creates two choices:

```
less than the key  
greater than the key
```

A multi-way node can create more choices because it may store more than one key.

Example node:

```
[30, 50]
```

This node creates three choices:

```
values less than 30  
values between 30 and 50  
values greater than 50
```

2.2 What does degree mean?

The **degree** of a multi-way tree means the maximum number of children a node can have.

A 2-3 tree has degree 3.

That means:

```
A 2-3 tree node can have at most 3 children.
```

Since a search-tree node with 3 children needs 2 separator keys, a 2-3 tree node can store at most 2 keys.

3. What is a 2-3 Tree?

A **2-3 tree** is a balanced multi-way search tree.

It is called a 2-3 tree because each internal node can behave like either:

- a 2-node
- a 3-node

A 2-3 tree is also a simple form of a B-tree of degree 3.

3.1 Main rules of a 2-3 tree

A 2-3 tree follows these important rules:

1. A node may store **1 key** or **2 keys**.
2. A node with 1 key has up to **2 children**.
3. A node with 2 keys has up to **3 children**.
4. Keys inside a node must always be sorted.
5. Duplicate keys are not allowed.
6. All leaves must be at the same level.
7. Insertion happens at a leaf.
8. If a node temporarily gets 3 keys, it must split.
9. Deletion must never leave a node empty.
10. If deletion creates an empty node, borrow first and merge second.

4. 2-Node and 3-Node

4.1 2-node

A **2-node** has:

1 key
2 child ranges

Example:

[30]

The search rule is:

values less than 30 go left
values greater than 30 go right

This is similar to a normal BST node.

4.2 3-node

A **3-node** has:

```
2 keys  
3 child ranges
```

Example:

```
[30, 50]
```

The search rule is:

```
values less than 30 go left  
values between 30 and 50 go middle  
values greater than 50 go right
```

So a 3-node can make three decisions instead of two.

4.3 Important key range rule

If a node has two keys:

```
[K1, K2]
```

then the keys must be sorted:

```
K1 < K2
```

The child ranges are:

```
child[0]: values less than K1  
child[1]: values greater than K1 and less than K2  
child[2]: values greater than K2
```

Example:

[30, 50]

The ranges are:

```
child[0]: values < 30
child[1]: 30 < values < 50
child[2]: values > 50
```

5. Node Structure in C++

The basic C++ structure for a 2-3 tree node is:

```
struct Node {
    int keys[2];           // Holds up to 2 keys
    Node* child[3];        // child[0], child[1], child[2]
    int keyCount;          // Number of keys currently stored
    bool isLeaf;           // True if the node has no children

    Node() {
        keyCount = 0;
        isLeaf = true;
        child[0] = child[1] = child[2] = nullptr;
    }
};
```

5.1 Meaning of each field

keys[2]

This array stores the keys inside the node.

A 2-3 node can store at most two keys, so the array size is 2.

Example:

```
keys[0] = 20
keys[1] = 40
```

This represents:

```
[20, 40]
```

`child[3]`

This array stores pointers to child nodes.

A 2-3 node can have at most three children, so the array size is 3.

The children represent:

```
child[0] = left child  
child[1] = middle child  
child[2] = right child
```

`keyCount`

This tells us how many keys are currently inside the node.

Possible values are:

```
1 or 2
```

A temporary overflow may create 3 keys during insertion, but a real 2-3 node cannot permanently have 3 keys.

`isLeaf`

This tells us whether the node is a leaf.

A leaf is a node with no children.

Insertion normally happens at a leaf.

`nullptr`

A child pointer is set to `nullptr` when that child does not exist.

Example:

```
child[0] = child[1] = child[2] = nullptr;
```

This means the node currently has no children.

6. Searching in a 2-3 Tree

Searching in a 2-3 tree is similar to searching in a BST, but a node may have more than one key.

6.1 Search in a 2-node

Node:

[30]

Search rules:

If target < 30, go left.
If target > 30, go right.
If target == 30, found it.

6.2 Search in a 3-node

Node:

[30, 50]

Search rules:

If target < 30, go left.
If 30 < target < 50, go middle.
If target > 50, go right.
If target == 30 or target == 50, found it.

6.3 Example searches

Given this node:

[30, 50]

Search for 20:

20 < 30
Go left.

Search for 40 :

30 < 40 < 50
Go middle.

Search for 70 :

70 > 50
Go right.

7. Balance Rule: All Leaves Stay at the Same Level

The most important balance rule is:

All leaves in a 2-3 tree must be at the same level.

7.1 Valid 2-3 tree

```
      [30]
      /
    [10] [40, 50]
```

Both leaves are on the same level.

This is valid.

7.2 Invalid 2-3 tree

```
      [30]
      /
    [10] [40]
           [50]
```

The right side is deeper than the left side.

This is invalid.

7.3 Why this rule matters

This rule prevents the tree from becoming a long chain.

Because all leaves stay level, the tree height stays small.

That is why search, insert, and delete stay fast.

8. Insertion in a 2-3 Tree

Insertion begins like BST insertion.

First, we search downward to find the correct leaf.

Then we insert into that leaf.

There are two main cases:

1. The leaf has space.
 2. The leaf is full and must split.
-

9. Insertion Case 1: Leaf Has Space

A 2-3 node can hold two keys.

So if the leaf has only one key, we can insert the new key directly.

Example:

Insert 20:

[20]

Insert 30:

[20, 30]

The node is valid because it has two keys.

No split is needed.

10. Insertion Case 2: Leaf is Full

If a leaf already has two keys, it is full.

Example:

```
[20, 30]
```

Now insert `40`.

Temporarily, we get:

```
[20, 30, 40]
```

But a 2-3 node cannot permanently hold three keys.

So we split.

10.1 Split rule

When a node has three keys temporarily:

```
smallest key stays in the left node  
middle key moves up to the parent  
largest key goes into the right node
```

Example:

```
20, 30, 40
```

Split result:

```
    [30]
   /
[20]  [40]
```

The middle key `30` moves upward.

10.2 Why the middle key moves up

The middle key separates the smaller key from the larger key.

For:

20, 30, 40

`30` is the best separator because:

20 < 30 < 40

So `30` becomes the parent, `20` goes left, and `40` goes right.

11. Conceptual Split Function

The curriculum gives this conceptual idea:

```
void splitNode(Node* parent, Node* fullNode) {
    // 1. Identify the 3 keys: 2 old keys plus the new key.
    // 2. Sort the 3 keys.
    // 3. The smallest key stays in the left node.
    // 4. The largest key goes to a new right node.
    // 5. The middle key moves UP to the parent.
    // If the parent is also full, the parent splits too.
}
```

In simple words:

When a node becomes too full, break it into two smaller nodes.
Move the middle key upward.

12. Insertion Dry Run: 20, 30, 40, 50, 60, 10, 15, 70, 80, 90

This dry run shows the important insertion ideas.

Step 1: Insert 20

The tree is empty, so `20` becomes the root.

```
[20]
```

Step 2: Insert 30

The root has space.

```
[20, 30]
```

Step 3: Insert 40

The root is full.

Temporary overflow:

```
[20, 30, 40]
```

Split:

```
    [30]
   /
[20] [40]
```

This is the first split.

The tree grows upward.

Step 4: Insert 50

`50 > 30`, so go right.

Right leaf is [40], which has space.

```
    [30]
   /
[20] [40, 50]
```

Step 5: Insert 60

60 > 30, so go right.

Right leaf [40, 50] is full.

Temporary overflow:

```
[40, 50, 60]
```

Split:

```
40 stays left
50 moves up
60 goes right
```

Root receives 50:

```
    [30, 50]
   /  |
[20] [40] [60]
```

Step 6: Insert 10

10 < 30, so go left.

Left leaf is [20], which has space.

```
    [30, 50]
   /  |
[10,20] [40] [60]
```

Step 7: Insert 15

15 < 30, so go left.

Left leaf [10, 20] is full.

Temporary overflow:

[10, 15, 20]

Split:

10 stays left
15 moves up
20 goes right

But the parent is already full:

[30, 50]

The parent receives 15 temporarily:

[15, 30, 50]

Now the parent must also split.

Split parent:

15 stays left
30 moves up
50 goes right

Result:

```
      [30]
      /
    [15]  [50]
    /  \  /
  [10] [20] [40] [60]
```

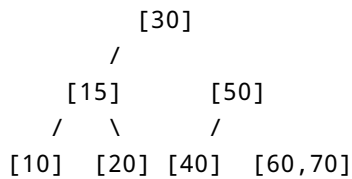
This is called a cascading split.

Step 8: Insert 70

70 > 30, go right.

70 > 50, go right.

Leaf [60] has space.



Step 9: Insert 80

Go to leaf [60, 70].

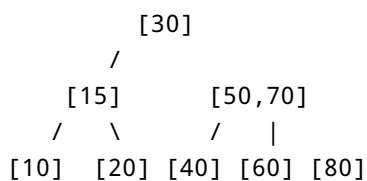
Temporary overflow:

[60, 70, 80]

Split:

60 stays left
70 moves up
80 goes right

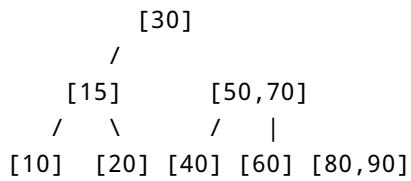
Parent [50] receives 70:



Step 10: Insert 90

Go to rightmost leaf [80].

It has space.



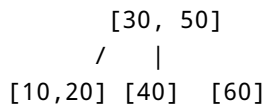
The tree is still balanced.

All leaves are still at the same level.

13. Cascading Split

A **cascading split** happens when a split pushes a key into a parent that is already full.

Example:



Insert **15** into **[10, 20]**.

Temporary leaf overflow:

[10, 15, 20]

Split the leaf:

10 stays left
15 moves up
20 goes right

But the parent is already full:

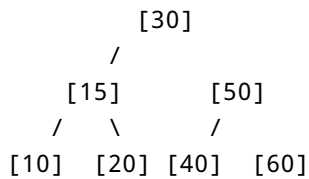
[30, 50]

So the parent becomes temporarily:

[15, 30, 50]

Now the parent also splits.

Final result:



Beginner rule:

A split can move upward. If the parent overflows, split the parent too.

14. Deletion in a 2-3 Tree

Deletion is more difficult than insertion because a node cannot be left empty.

A valid 2-3 tree node must have at least one key unless it is a special case during repair.

14.1 Main deletion rule

If deletion makes a node empty, fix it using this priority:

1. Borrow first.
2. Merge second.

This means:

If a sibling has extra keys, borrow one.
If no sibling has extra keys, merge nodes.

15. Simple Leaf Deletion

If the key is in a leaf with two keys, deletion is easy.

Example:

```
[80, 90]
```

Delete 90 :

```
[80]
```

This is valid because the node still has one key.

No borrow or merge is needed.

16. Empty Node Problem

If deletion removes the only key in a node, the node becomes empty.

Example:

```
[80]
```

Delete 80 :

```
[]
```

This is not allowed.

So the tree must repair itself.

The repair choices are:

```
Borrow from a sibling if possible.  
Otherwise merge with a sibling.
```

17. Borrowing

Borrowing happens when a sibling has two keys and can spare one.

17.1 Borrow from right sibling

Example:

```
    [70]
   /
  []    [80, 90]
```

The left child is empty.

The right sibling has two keys, so it can lend one.

Borrowing steps:

1. Parent key 70 moves down into the empty child.
2. Right sibling key 80 moves up to the parent.
3. Right sibling keeps 90.

Result:

```
    [80]
   /
  [70]  [90]
```

17.2 Borrow from left sibling

Borrowing from the left sibling is similar.

The idea is:

1. Parent key moves down into the empty child.
2. Left sibling sends one key up to the parent.
3. Left sibling loses that extra key but remains valid.

17.3 Why borrowing is preferred

Borrowing is preferred because it fixes the empty node without shrinking the tree.

It keeps the structure more stable.

18. Merging

Merging happens when borrowing is not possible.

Borrowing is not possible when siblings have only one key each.

Example:

```
    [70]
   /
[60]  []
```

The right child is empty.

The left sibling `[60]` has only one key, so it cannot lend.

So we merge:

```
Bring parent key 70 down.
Combine it with sibling 60.
Remove the empty child.
```

Result:

```
[60, 70]
```

18.1 Important note about merging

Merging can make the parent lose a key.

If the parent becomes empty, repair may move upward.

So deletion repairs can also cascade upward, just like insertion splits can cascade upward.

19. Conceptual Deletion Repair Function

The curriculum gives this idea:

```

void fixEmptyNode(Node* parent, int childIndex) {
    // Attempt 1: Borrow from Right Sibling
    if (rightSiblingHasTwoKeys()) {
        // Bring parent key down, push sibling's first key up.
        borrowFromRight();
        return;
    }

    // Attempt 2: Borrow from Left Sibling
    if (leftSiblingHasTwoKeys()) {
        // Bring parent key down, push sibling's last key up.
        borrowFromLeft();
        return;
    }

    // Attempt 3: Merge if borrowing fails
    // Combine empty node + parent key + sibling into one node.
    mergeNodes();
}

```

In simple words:

First try to borrow from the right sibling.
 If that is not possible, try to borrow from the left sibling.
 If borrowing is impossible, merge.

20. Deletion Dry Run Cases

Case 1: Delete 90

Suppose a leaf is:

[80, 90]

Delete 90:

[80]

This is simple because the node still has one key.

Case 2: Delete 80

Suppose deleting 80 leaves an empty node.

If the sibling cannot lend, merge.

Example idea:

```
Sibling: [60]
Parent key: 70
Empty node: []
```

Merge:

```
[60, 70]
```

Case 3: Delete 60 with borrowing

Suppose deleting 60 leaves an empty node, but the right sibling has two keys:

```
Right sibling: [80, 90]
Parent key: 70
Empty node: []
```

Borrow:

```
70 moves down
80 moves up
sibling becomes [90]
```

Result:

```
    [80]
   /
[70] [90]
```

Case 4: Delete 20 with merging

Suppose deleting 20 leaves an empty node and siblings cannot lend.

Then merge with a sibling and a parent key.

Example idea:

```
Left sibling: [10]
Parent key: 15
Empty node: []
```

Merge:

```
[10, 15]
```

If the parent becomes empty, the repair continues upward.

Case 5: Delete 50 from an internal node

If 50 is inside an internal node, do not simply remove it.

Instead:

1. Find predecessor or successor.
2. Replace 50 with that value.
3. Delete the predecessor or successor from the leaf.
4. Repair the leaf if needed.

Example:

```
      [30, 50]
     /   |
    [20] [40] [60]
```

To delete 50, use predecessor 40:

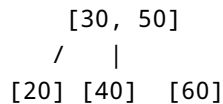
```
Replace 50 with 40.
Then delete the old 40 from the leaf.
```

21. Internal Deletion

21.1 What is an internal key?

An internal key is a key stored in a node that is not a leaf.

Example:



Here, **30** and **50** are internal keys because they are in the root, and the root has children.

21.2 Why not delete internal keys directly?

If we directly remove an internal key, we may break the search structure.

So we replace it with a nearby value from a leaf.

The two common choices are:

predecessor
successor

21.3 Predecessor

The predecessor is the largest key smaller than the key being deleted.

Example:

Predecessor of 50 is 40.

21.4 Successor

The successor is the smallest key larger than the key being deleted.

Example:

Successor of 50 might be 60.

21.5 Internal deletion rule

The rule is:

Replace the internal key with predecessor or successor.
Then physically delete that replacement key from a leaf.

This keeps the tree order correct.

22. Complexity

Because all leaves stay at the same level, the height of the tree stays small.

The height grows logarithmically as the number of keys grows.

That means operations are fast.

Operation	Time Complexity	Simple Meaning
Search	$O(\log n)$	We only travel down the height of the tree.
Insert	$O(\log n)$	We search down, then maybe split upward.
Delete	$O(\log n)$	We search down, then maybe repair upward.
Storage	$O(n)$	We store all keys in nodes.

22.1 Why this is better than a bad BST

A bad BST can become a chain:

$O(n)$

A 2-3 tree stays balanced:

$O(\log n)$

So 2-3 trees are more reliable for search operations.

23. Why 2-3 Trees Matter

2-3 trees are important because they introduce the idea behind B-trees and B+ trees.

B-trees and B+ trees are used in databases and file systems.

The reason is simple:

A node with multiple keys gives more information at once, so the tree can be wider and shorter.

Shorter trees mean fewer steps during search.

This matters especially when data is stored on disk, because reading from disk is slower than reading from memory.

24. Complete Teaching Program for Phase 1

This program is a teaching demo. It does not implement a full production 2-3 tree. Instead, it demonstrates the main ideas:

- splitting
- cascading split idea
- borrowing
- merging
- internal deletion idea

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>
using namespace std;

struct Node23 {
    vector<int> keys; // Teaching version of keys[2]

    explicit Node23(initializer_list<int> init = {}) : keys(init) {
        sort(keys.begin(), keys.end());
    }
};

void printNode(const string& name, const Node23& node) {
```

```

    cout << name << " = [";
    for (size_t i = 0; i < node.keys.size(); ++i) {
        if (i) cout << ", ";
        cout << node.keys[i];
    }
    cout << "]\n";
}

void splitThreeKeys(vector<int> keys) {
    sort(keys.begin(), keys.end());

    cout << "Splitting overflow keys: ";
    for (int x : keys) cout << x << " ";
    cout << "\n";

    int leftKey = keys[0];
    int middleUp = keys[1];
    int rightKey = keys[2];

    cout << "Left node keeps: [" << leftKey << "]\n";
    cout << "Middle key moves UP: " << middleUp << "\n";
    cout << "Right node gets: [" << rightKey << "]\n\n";
}

void borrowFromRight(Node23& emptyChild, int& parentKey, Node23& rightSibling) {
    cout << "Borrow from right sibling\n";
    cout << "Parent key moves down: " << parentKey << "\n";

    emptyChild.keys.push_back(parentKey);

    parentKey = rightSibling.keys.front();
    rightSibling.keys.erase(rightSibling.keys.begin());

    cout << "Sibling first key moves up to parent: " << parentKey << "\n";
}

void mergeWithSibling(Node23& leftSibling, int parentKey, Node23& emptyChild) {
    cout << "Merge because no sibling can spare a key\n";

    // emptyChild is conceptually removed after merge
    (void)emptyChild;

    leftSibling.keys.push_back(parentKey);
    sort(leftSibling.keys.begin(), leftSibling.keys.end());
}

int main() {
    cout << "Phase 1: 2-3 Tree Teaching Demo\n\n";
}

```

```

    cout << "Insertion split demo: insert 40 into [20, 30]\n";
    splitThreeKeys({20, 30, 40});

    cout << "Cascading split idea: insert 15 into leaf [10, 20], then parent
[30, 50] overflows\n";
    splitThreeKeys({10, 15, 20});

    cout << "Now parent would receive 15, becoming [15, 30, 50], so split again:
\n";
    splitThreeKeys({15, 30, 50});

    cout << "Deletion borrow demo: empty child, parent key 70, right sibling
[80, 90]\n";
    Node23 emptyChild;
    int parentKey = 70;
    Node23 rightSibling{80, 90};

    borrowFromRight(emptyChild, parentKey, rightSibling);

    printNode("Filled child", emptyChild);
    cout << "Parent key is now " << parentKey << "\n";
    printNode("Right sibling", rightSibling);
    cout << "\n";

    cout <<
    "Deletion merge demo: delete 80; merge left sibling [60] with parent key 70\n";
    Node23 leftSibling{60};
    Node23 removedEmptyChild;

    mergeWithSibling(leftSibling, 70, removedEmptyChild);
    printNode("Merged node", leftSibling);

    cout << "\nInternal deletion idea: replace an internal key with predecessor
or successor,\n";
    cout << "then physically delete that borrowed key from a leaf.\n";

    return 0;
}

```

24.1 What the program teaches

Node23

This is a simple teaching node.

It uses a vector instead of fixed arrays so the demo is easier to read.

splitThreeKeys

This shows the split rule:

```
smallest stays left  
middle moves up  
largest goes right
```

borrowFromRight

This shows borrowing:

```
parent key moves down  
sibling key moves up
```

mergeWithSibling

This shows merging:

```
parent key moves down  
sibling and empty node combine
```

25. Common Beginner Mistakes

Mistake 1: Thinking a 2-3 node can permanently hold 3 keys

A 2-3 node can hold only one or two keys.

Three keys are only a temporary overflow during insertion.

Wrong:

```
[10, 20, 30]
```

Correct after split:

```
    [20]
   /
[10]  [30]
```

Mistake 2: Forgetting keys inside a node must be sorted

Wrong:

```
[50, 30]
```

Correct:

```
[30, 50]
```

If the keys are not sorted, we cannot correctly choose left, middle, or right.

Mistake 3: Deleting an internal key directly

If the key is inside an internal node, do not remove it directly.

Use predecessor or successor first.

Mistake 4: Merging before borrowing

The repair order is:

1. Borrow first.
2. Merge second.

Do not merge if borrowing is possible.

Mistake 5: Ignoring cascading splits

A split may push a key into a parent.

If that parent becomes full, it must split too.

Mistake 6: Ignoring cascading deletion repair

A merge may make the parent empty.

If the parent becomes empty, deletion repair may continue upward.

Mistake 7: Forgetting that all leaves must stay level

A 2-3 tree is not valid if one leaf is deeper than another.

All leaves must be at the same level.

26. Key Vocabulary

Binary Search Tree

A tree where each node stores one key and has at most two children.

Multi-way search tree

A search tree where a node can have more than two children.

Degree

The maximum number of children a node can have.

2-3 tree

A balanced multi-way search tree where each node can be a 2-node or a 3-node.

2-node

A node with one key and two child ranges.

3-node

A node with two keys and three child ranges.

Leaf

A node with no children.

Root

The top node of the tree.

Split

Breaking an overflowing node into two nodes and moving the middle key upward.

Cascading split

A split that causes the parent to overflow, forcing another split upward.

Borrow

A deletion repair where a sibling gives one key through the parent.

Merge

A deletion repair where an empty node, a parent key, and a sibling combine.

Predecessor

The largest key smaller than a given key.

Successor

The smallest key larger than a given key.

$O(\log n)$

A fast time complexity where the number of steps grows slowly as input grows.

27. Phase 1 Final Summary

A 2-3 tree is a balanced search tree.

It improves on a normal BST by allowing a node to store one or two keys.

A 2-3 tree node can be:

2-node: 1 key, 2 children
3-node: 2 keys, 3 children

The keys inside a node must be sorted.

A 3-node like this:

```
[30, 50]
```

creates three ranges:

```
less than 30  
greater than 30 and less than 50  
greater than 50
```

The most important balance rule is:

```
All leaves must stay at the same level.
```

Insertion happens at a leaf.

If a node temporarily gets three keys, split it:

```
smallest stays left  
middle moves up  
largest goes right
```

Deletion must never leave a node empty.

If deletion creates an empty node:

```
Borrow first.  
Merge second.
```

If deleting an internal key, replace it with its predecessor or successor, then delete from a leaf.

Because the tree stays balanced:

```
Search:  $O(\log n)$   
Insert:  $O(\log n)$   
Delete:  $O(\log n)$   
Storage:  $O(n)$ 
```

28. Mini Practice Questions

Question 1

Insert:

20, 30, 40

What is the final tree?

Answer:

```
      [30]
     /
    [20] [40]
```

Reason:

[20, 30, 40] overflows.
30 moves up.

Question 2

In this node:

[30, 50]

Where does 40 go?

Answer:

middle child

Reason:

30 < 40 < 50

Question 3

A node becomes empty after deletion. Its sibling has two keys. What should we do?

Answer:

Borrow.

Reason:

Borrow before merging.

Question 4

Can a 2-3 tree node permanently hold three keys?

Answer:

No.

Reason:

Three keys are only temporary before a split.

Question 5

Why are 2-3 trees faster than bad BSTs?

Answer:

Because 2-3 trees stay balanced.
All leaves stay at the same level.
So operations stay $O(\log n)$.

Question 6

What happens if a split pushes a key into a full parent?

Answer:

The parent also splits.

This is called a cascading split.

Question 7

When deleting an internal key, what should we do first?

Answer:

Replace it with predecessor or successor.
Then delete the replacement key from a leaf.

29. One-Page Memory Sheet

2-3 Tree = balanced multi-way search tree

Node types:

2-node = 1 key, 2 children

3-node = 2 keys, 3 children

Main ordering:

[K1, K2]

left: values < K1

middle: K1 < values < K2

right: values > K2

Important rules:

- Keys inside a node stay sorted.
- No duplicate keys.
- All leaves stay at the same level.
- Insert into leaves.
- 3 keys means temporary overflow.
- Overflow causes split.
- Middle key moves up.
- Deletion cannot leave empty nodes.
- Borrow first.
- Merge second.
- Internal deletion uses predecessor or successor.

Complexity:

```
Search =  $O(\log n)$   
Insert =  $O(\log n)$   
Delete =  $O(\log n)$   
Storage =  $O(n)$ 
```

End of Phase 1 Notes